

```
from google.colab import files
uploaded = files.upload()

Choose Files | advertising...les_data.csv
advertising_sales_data.csv(text/csv) - 5555 bytes, last modified: 3/31/2026 - 100% done
Saving advertising_sales_data.csv to advertising_sales_data.csv
```

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Load dataset
df = pd.read_csv('Facebook_Marketplace_data.csv')
# Fix datetime
df['status_published'] = pd.to_datetime(
    df['status_published'],
    format='mixed',
    dayfirst=True
)

# Feature: hour
df['hour'] = df['status_published'].dt.hour

# X (input) and y (output)
X = df[['hour']]
y = df['num_reactions']

# Train model
model = LinearRegression()
model.fit(X, y)

# Coefficient (impact of time)
print("Impact of hour on reactions:", model.coef_[0])

Impact of hour on reactions: 1.1430589561726807
```

CODE 2

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df = pd.read_csv('Facebook_Marketplace_data.csv')

# features and target
X = df[['num_comments', 'num_shares']]
y = df['num_reactions']

# model
model = LinearRegression()
model.fit(X, y)

# coefficients
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
print("R^2 Score:", model.score(X, y))

Coefficients: [-0.00862541  0.91874266]
Intercept: 195.2818991541085
R^2 Score: 0.06302398816420274
```

CODE 3

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.cluster import KMeans

df = pd.read_csv('Facebook_Marketplace_data.csv')

data = df[['status_type', 'num_reactions', 'num_comments', 'num_shares',
            'num_likes', 'num_loves', 'num_wows', 'num_hahas', 'num_sads', 'num_angrys']].copy()

data['status_type'] = LabelEncoder().fit_transform(data['status_type'])

kmeans = KMeans(n_clusters=3, random_state=0)
data['cluster'] = kmeans.fit_predict(data)

# check cluster centers
print(kmeans.cluster_centers_)

[[1.66884430e+00 1.13000000e+02 1.08951003e+02 2.47901695e+01
 1.03412350e+02 8.33551097e+00 5.78783637e-01 4.09706020e-01
 1.82454503e-01 7.82392285e-02]
 [2.98540146e+00 4.86737226e+02 5.30959124e+03 4.80408759e+02
 3.76072993e+02 9.50729927e+01 4.80291971e+00 7.39416058e+00
 2.51824818e+00 8.75912409e-01]
 [1.82231405e+00 1.71315289e+03 3.17871901e+02 1.17700413e+02
 1.65226033e+03 4.77747934e+01 9.73347107e+00 2.60950413e+00
 4.13223140e-01 3.61570248e-01]]
```

CODE 4

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

df = pd.read_csv('Facebook_Marketplace_data.csv')

data = df[['status_type', 'num_reactions', 'num_comments', 'num_shares',
            'num_likes', 'num_loves', 'num_wows', 'num_hahas', 'num_sads', 'num_angrys']].copy()

data['status_type'] = LabelEncoder().fit_transform(data['status_type'])

# Elbow method
wcss = []
for i in range(1, 10):
    kmeans = KMeans(n_clusters=i, random_state=0)
    kmeans.fit(data)
    wcss.append(kmeans.inertia_)

plt.plot(range(1,10), wcss)
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.title('Elbow Method')
plt.show()
```

Elbow Method

Number of Clusters	WCSS (approx.)
1	8.5
2	5.0
3	2.8
4	2.2
5	1.8
6	1.5
7	1.2
8	1.0
9	0.8

CODE 5

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv('Facebook_Marketplace_data.csv')

# encode status_type
le = LabelEncoder()
df['status_type_encoded'] = le.fit_transform(df['status_type'])

# count original types
print(df['status_type'].value_counts())

status_type
photo      4288
video     2334
```

```
status      365
link         63
Name: count, dtype: int64
```

CODE 6

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv('Facebook_Marketplace_data.csv')

# encode post type (ML preprocessing)
df['status_type'] = LabelEncoder().fit_transform(df['status_type'])

# calculate averages
result = df.groupby('status_type')[['num_reactions', 'num_comments', 'num_shares']].mean()

print(result)
```

	num_reactions	num_comments	num_shares
status_type			
0	370.142857	5.698413	4.396825
1	181.290345	15.993470	2.553871
2	438.783562	36.238356	2.558904
3	283.409597	642.478149	115.679949

Below are the answers for Sales prediction Dataset

CODE 1

```
import pandas as pd

df = pd.read_csv('advertising_sales_data.csv' , encoding='latin1' , on_bad_lines='skip')

# average TV advertising spend
print(df['TV'].mean())
```

147.0425

CODE 2

```
import pandas as pd
from sklearn.linear_model import LinearRegression
df = pd.read_csv('advertising_sales_data.csv' , encoding='latin1' , on_bad_lines='skip')
df = df[['Radio','Sales']].dropna()
X = df[['Radio']]
y = df['Sales']
model = LinearRegression()
model.fit(X,y)
print("Coefficient:", model.coef_[0])
print("R^2 Score:", model.score(X, y))
```

Coefficient: 0.12443922619598301
R^2 Score: 0.12230947318480201

CODE 3

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df = pd.read_csv('advertising_sales_data.csv', encoding='latin1', engine='python')

# select columns and remove missing values
df = df[['TV', 'Radio', 'Newspaper', 'Sales']].dropna()

X = df[['TV', 'Radio', 'Newspaper']]
y = df['Sales']

# train model
model = LinearRegression()
model.fit(X, y)

# coefficients
print("TV:", model.coef_[0])
print("Radio:", model.coef_[1])
print("Newspaper:", model.coef_[2])
```

TV: 0.054494075273967185
Radio: 0.10718024415921726
Newspaper: -1.9019018071206317e-05

“Radio advertising has the highest impact on sales, as it has the largest positive coefficient. TV also has a positive but smaller impact, while Newspaper has negligible impact on sales.”

CODE 4

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

df = pd.read_csv('advertising_sales_data.csv', encoding='latin1', engine='python')

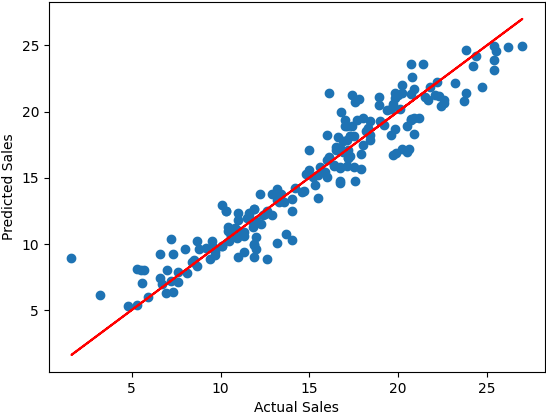
df = df[['TV', 'Radio', 'Newspaper', 'Sales']].dropna()

X = df[['TV', 'Radio', 'Newspaper']]
y = df['Sales']

model = LinearRegression()
model.fit(X, y)

y_pred = model.predict(X)

# plot
plt.scatter(y, y_pred)
plt.plot(y, y, color='red') # regression reference line
plt.xlabel('Actual Sales')
plt.ylabel('Predicted Sales')
plt.title('Actual vs Predicted Sales')
plt.show()
```



CODE 5

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df = pd.read_csv('advertising_sales_data.csv', encoding='latin1', engine='python')

# prepare data
df = df[['TV', 'Radio', 'Newspaper', 'Sales']].dropna()

X = df[['TV', 'Radio', 'Newspaper']]
y = df['Sales']

# train model
model = LinearRegression()
model.fit(X, y)

# new data prediction
new_data = pd.DataFrame([[200, 40, 50]], columns=['TV', 'Radio', 'Newspaper'])
predicted_sales = model.predict(new_data)
```

print("Predicted Sales:", predicted_sales[0])
Predicted Sales: 19.819378160410906

CODE 6

<pre>import pandas as pd from sklearn.model_selection import train_test_split from sklearn.linear_model import LinearRegression from sklearn.metrics import mean_squared_error # Load dataset df = pd.read_csv('advertising_sales_data.csv', encoding='latin1', engine='python') # replace with your file name # Check missing values print("Missing values:\n", df.isnull().sum()) # Fill NaN values with mean (imputation) df = df.fillna(df.mean(numeric_only=True)) # Define features and target X = df[['TV', 'Radio', 'Newspaper']] # change column names if needed y = df['Sales'] # target column # Split dataset X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # Train model model = LinearRegression() model.fit(X_train, y_train) # Predict y_pred = model.predict(X_test) # Evaluate mse = mean_squared_error(y_test, y_pred) print("Mean Squared Error:", mse) print("Coefficients:", model.coef_)</pre>
Missing values: Campaign 0 TV 0 Radio 2 Newspaper 0 Sales 0 dtype: int64 Mean Squared Error: 2.9074318865000555 Coefficients: [0.05450899 0.10096678 0.00431997]

With normalization

<pre>from sklearn.preprocessing import StandardScaler scaler = StandardScaler() X_train_scaled = scaler.fit_transform(X_train) X_test_scaled = scaler.transform(X_test) model2 = LinearRegression() model2.fit(X_train_scaled, y_train) y_pred2 = model2.predict(X_test_scaled) print("MSE with normalization:", mean_squared_error(y_test, y_pred2)) print("Coefficients with normalization:", model2.coef_)</pre>
MSE with normalization: 2.9074318865000572 Coefficients with normalization: [4.58718413 1.49013705 0.08757788]

CODE 7

<pre>import pandas as pd from sklearn.model_selection import train_test_split from sklearn.linear_model import LinearRegression from sklearn.metrics import mean_squared_error # Load dataset df = pd.read_csv('advertising_sales_data.csv', encoding='latin1', engine='python') # replace with your file name # Handle NaN values (important) df = df.fillna(df.mean(numeric_only=True)) # Use only Radio and Newspaper as predictors X = df[['Radio', 'Newspaper']] y = df['Sales'] # Split data X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # Train model model = LinearRegression() model.fit(X_train, y_train) # Predict y_pred = model.predict(X_test) # Evaluate mse = mean_squared_error(y_test, y_pred) print("MSE (Radio & Newspaper only):", mse) print("Coefficients:", model.coef_)</pre>
MSE (Radio & Newspaper only): 27.50240015808231 Coefficients: [0.11797449 0.00383138]